

Flutter

Dosen	: IKetut Gunawan, S.KOM, M.T.I
Kode/Mata Kuliah	: KB1185/ Aplikasi Mobile
Bobot Mata Kuliah	: 3 Sks
Prodi	: Teknik Informatika
Pertemuan	: 2

Cakupan Materi

-

Tujuan Pembelajaran

Rencana Aktivitas Kelas (150 menit)

Waktu	Aktivitas

0

Check Installation dan Version

```
$> flutter doctor
```

Lihat hasil pengecekan flutter doctor installation mana yang belum ter install

```
$> flutter version
```

```
$> flutter devices
```

```
$> flutter emulator
```

1

Create New Project Flutter

```
$> flutter create nama_project
```

Perhatikan hasil dari membuat project apa yang di hasilkan :

```
$ cd nama_project  
$ flutter run
```

Sebelum lakukan git add lakukan clean hasil build dahulu

```
$ flutter clean
```

Baru lakukan git

```
$nama_project> git init  
$nama_project> git add .  
$nama_project> git commit -m "Inital project "  
$nama_project> git push
```

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(  
    const MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Text(  
            'Hello World!',  
            //style: TextStyle(fontSize: 24),  
          ),  
        ),  
      ),  
    ),  
  );  
}
```

Membedah Kode (Konsep Lego)

- **import 'package:flutter/material.dart';**

Ini ibarat Kita **membuka kotak peralatan**. Kita memanggil semua komponen desain stKitar (seperti tombol, warna, bentuk dasar) yang sudah disediakan oleh Google agar bisa Kita pakai.

- **void main() { runApp(...) }**

Ini adalah **tombol power**. Setiap kali aplikasi dibuka, sistem akan mencari fungsi **main()** terlebih

dahulu untuk menjalankan (**runApp**) keseluruhan aplikasi Kita.

- **MaterialApp**

Ini adalah **papan alas Lego** yang paling besar. Komponen ini membungkus seluruh aplikasi Kita dan menyiapkan pengaturan dasar, seperti sistem warna dan gaya desain.

- **Scaffold**

Ini adalah **kerangka halaman kosong**. **Scaffold** memberi Kita kanvas berlatar belakang putih tempat Kita bisa menempelkan teks, tombol, atau gambar.

- **Center**

Sesuai namanya, ini adalah alat bantu penataan yang memastikan apa pun yang ada di dalamnya posisinya berada **tepat di tengah-tengah** layar.

- **Text**

Ini adalah **blok Lego berbentuk tulisan**. Di sini kita memasukkan kata 'Hello World!' dan sedikit mengatur ukurannya menggunakan **TextStyle(fontSize: 24)** agar teks tidak terlalu kecil.

Perhatikan bagaimana **Text** dibungkus oleh **Center**, **Center** dibungkus oleh **Scaffold**, dan **Scaffold** dibungkus oleh **MaterialApp**. Inilah yang disebut prinsip "Widget di dalam Widget" pada Flutter.

1

Membuat Widget Buatan Sendiri (Custom Widget)

```
void main() {  
  // runApp(  
  //   const MaterialApp(  
  //     home: Scaffold(body: Center(child: Text('Hello, World!'))),  
  //   ),  
  // );  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(body: Center(child: Text('Hello, World!'))),  
    );  
  }  
}
```

Mari kita gunakan analogi Lego kembali.

Pada kode lama (yang dimatikan/di-comment), membuat seluruh bangunan Lego langsung di atas tombol saklar utama (main).

Tapi bayangkan jika aplikasinya sudah sebesar Gopay; saklar utamanya akan tertumpuk jutaan baris kode yang sangat berantakan.

Pada Kode kita yang baru kita merakit bangunannya di "meja kerja terpisah" yang Kita beri nama MyApp. Lalu di saklar utama (main), Kita tinggal memanggil nama bangunan tersebut.

Membedah Komponen Baru

1. class MyApp extends StatelessWidget

Ini adalah cara Kita mendeklarasikan: "Saya membuat cetakan blok Lego baru, namanya MyApp. Cetakan ini berjenis Stateless (tampilannya tetap, tidak berubah-ubah setelah digambar)."

2. Widget build(BuildContext context)

Ini adalah "pabrik" atau instruksi pembuatannya. Setiap kali Flutter ingin menampilkan MyApp, sistem akan masuk ke dalam blok build ini dan membaca apa yang harus di-return (dikembalikan/dihasilkan). Di sinilah Kita meletakkan susunan MaterialApp, Scaffold, dan Text tadi.

3. `runApp(const MyApp());`

Sekarang fungsi `main()` Kita menjadi sangat bersih dan rapi. Ia hanya bertugas menekan tombol mulai dan memanggil cetakan `MyApp` yang sudah Kita susun di bawahnya.

Mengapa Cara Ini Jauh Lebih Baik?

Mencegah "**Kode Spageti**": Ketika aplikasi Kita semakin canggih, Kita akan memiliki banyak halaman (halaman Login, BerKita, Pengaturan). Dengan memisahkan setiap halaman menjadi class tersendiri seperti ini, kode Kita tidak akan kusut seperti spageti.

Mudah Dicari: Jika nanti ada kesalahan (bug) pada teks "**Hello World**", Kita tahu persis harus mencarinya di dalam class `MyApp`, bukan mengaduk-aduk fungsi `main()`.

Bisa Dipakai Berulang: Jika Kita membuat class berisi desain tombol khusus yang cantik, Kita bisa memanggil nama class tersebut berulang kali di halaman mana pun tanpa harus mengetik ulang kode desainnya.

Apakah value dari "Hello World" kita bisa ganti jadi `const` ? . Dan apakah bisa dalam class

```
void main() {  
  // runApp(  
  //   const MaterialApp(  
  //     home: Scaffold(body: Center(child: Text('Hello, World!'))),  
  //   ),  
  // );  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  const hello = 'Hello, World!';  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(body: Center(child: Text(hello))),  
    );  
  }  
}
```

Jika const dibuat didalam class apakah akan menjadi error ?

Kenapa Error ?

Karena variabel biasa di dalam *class* itu sifatnya milik objek (*instance*).

Objek baru dibuat saat aplikasi berjalan.

Padahal, `const` menuntut nilainya harus sudah pasti *sejak sebelum aplikasi dijalankan* (pada waktu kompilasi).

Solusinya, tambahkan `static`

```
void main() {  
  // runApp(  
  //   const MaterialApp(  
  //     home: Scaffold(body: Center(child: Text('Hello, World!'))),  
  //   ),  
  // );  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  static const hello = 'Hello, World!';  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(body: Center(child: Text(hello))),  
    );  
  }  
}
```

Dengan menambahkan `static`, variabel tersebut menjadi milik kelas secara global, bukan milik objek. Ini membuat Dart tahu di mana harus menyimpan nilai `const` tersebut secara permanen di memori.

Di Dalam Fungsi atau Metode (Lokal)

Jika Anda mendeklarasikan variabelnya **di dalam sebuah fungsi** (misalnya di dalam fungsi `main()` atau fungsi `build`), Anda **tidak perlu** menggunakan `static`. Anda bahkan bisa langsung memakai `const` atau `final` secara bebas.

```
void main() {  
  // Ini TIDAK PERLU static, langsung pakai const bisa:  
  const String hello = 'Hello, World!';  
  
  // Atau pakai String biasa:  
  String pesan = 'Halo';  
  
  print(hello);  
}
```

Kenapa ini boleh? Karena variabel di dalam fungsi sifatnya lokal dan hanya hidup sementara saat fungsi itu dipanggil, sehingga tidak memerlukan aturan *class-level* seperti `static`.

Kesimpulan:

- Jika variabel dibuat **di dalam class** (di luar fungsi), dan Anda ingin membuatnya tetap (`const`), **wajib** ditambah `static`.
- Jika variabel dibuat **di dalam fungsi/metode**, Anda **tidak boleh** pakai `static`. Cukup gunakan `const` atau tipe data biasa seperti `String hello = ...`.

Contoh kode Dart sederhana yang menunjukkan perbedaan nyata antara variabel `static` dan **tidak static** (biasa):

```
class Siswa {
  // 1. Variabel STATIC (milik bersama / kelas)
  // Hanya ada 1 di memori untuk semua siswa
  static String namaSekolah = "SMA Merdeka";

  // 2. Variabel TIDAK STATIC (milik masing-masing objek)
  // Setiap siswa punya nama dan nilainya sendiri
  String namaSiswa;

  // Konstruktor untuk mengisi nama siswa
  Siswa(this.namaSiswa);

  // Fungsi biasa untuk menampilkan info
  void perkenalkanDiri() {
    print("Halo, nama saya $namaSiswa dan sekolah saya di $namaSekolah.");
  }
}

void main() {
  // --- CARA MENGAKSES STATIC ---
  // Kita bisa langsung memanggil variabel static lewat Nama Kelasnya
  // TANPA harus membuat objek siswa terlebih dahulu.
  print(Siswa.namaSekolah); // Output: SMA Merdeka

  // --- CARA MENGAKSES TIDAK STATIC ---
  // Kita HARUS membuat objek (instansiasi) terlebih dahulu menggunakan 'new' atau
  // memanggil konstruktor.
  Siswa siswa1 = Siswa("Budi");
  Siswa siswa2 = Siswa("Ani");

  siswa1.perkenalkanDiri(); // Output: Halo, nama saya Budi dan sekolah saya di SMA
  // Merdeka.
  siswa2.perkenalkanDiri(); // Output: Halo, nama saya Ani dan sekolah saya di SMA
  // Merdeka.

  // --- APA YANG TERJADI JIKA VARIABEL STATIC DIUBAH? ---
  // Karena sifatnya menempel pada kelas (dibagikan bersama),
  // jika kita mengubah nilai variabel static, semua objek akan ikut merasakannya.

  Siswa.namaSekolah = "SMA Harapan Bangsa";

  print("\n--- Setelah nama sekolah diubah ---");
  siswa1.perkenalkanDiri(); // Output: Halo, nama saya Budi dan sekolah saya di SMA
  // Harapan Bangsa.
  siswa2.perkenalkanDiri(); // Output: Halo, nama saya Ani dan sekolah saya di SMA
  // Harapan Bangsa.
}
```

Kesimpulan dari Contoh di Atas:

1. `namaSekolah` (**Static**): Cukup dibuat sekali, nilainya sama untuk semua objek, dan bisa diakses langsung via `Siswa.namaSekolah`. Jika diubah, semuanya ikut berubah.
2. `namaSiswa` (**Tidak Static**): Milik masing-masing objek (`siswa1` namanya Budi, `siswa2` namanya Ani). Nilainya independen antara satu objek dengan objek lainnya.

2

Widget

A. TextStyle dasar

Mulai dari yang paling sederhana.

```
class BasicTextWidget extends StatelessWidget {  
  const BasicTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Text(  
      'Hello Flutter',  
      style: TextStyle(  
        fontSize: 24,  
        color: Colors.blue,  
      ),  
    );  
  }  
}
```

Update `fontSize` dan `colors`

4. Font, Weight dan Style

```
class FontTextWidget extends StatelessWidget {  
  const FontTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        Text(  
          'Normal',  
          style: TextStyle(  
            fontSize: 20,  
          ),  
        ),  
  
        Text(  
          'Bold',  
          style: TextStyle(  
            fontSize: 20,  
            fontWeight: FontWeight.bold,  
          ),  
        ),  
  
        Text(  
          'Semi Bold',  
          style: TextStyle(  
            fontSize: 20,  
            fontWeight: FontWeight.w600,  
          ),  
        ),  
  
        Text(  

```

```
'Italic',  
style: TextStyle(  
  fontSize: 20,  
  fontStyle: FontStyle.italic,  
),  
,  
],  
);  
}  
}
```

Di sini mahasiswa mulai mengenal:

FontWeight.bold

FontWeight.w600

FontStyle.italic

5. Spacing

Ini bagus untuk eksplorasi karena mahasiswa bisa langsung melihat perbedaannya.

```
class SpacingTextWidget extends StatelessWidget {  
  const SpacingTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
  
        Text(  
          'Normal Text',  
          style: TextStyle(  
            fontSize: 20,  
          ),  
        ),  
  
        SizedBox(height: 20),  
  
        Text(  
          'Letter Spacing',  
          style: TextStyle(  
            fontSize: 20,  
            letterSpacing: 5,  
          ),  
        ),  
  
        SizedBox(height: 20),  
  
        Text(  
          'Word Spacing Example',  
          style: TextStyle(  
            fontSize: 20,
```

```
        wordSpacing: 10,  
      ),  
    ),  
  
    SizedBox(height: 20),  
  
    Text(  
      'Line 1\nLine 2\nLine 3',  
      style: TextStyle(  
        fontSize: 20,  
        height: 2,  
      ),  
    ),  
  ],  
);  
}  
}
```

Mahasiswa dapat mencoba:

letterSpacing: 5

menjadi:

letterSpacing: 1

atau:

height: 2

menjadi:

height: 1

6. Decoration dan Shadow

```
class DecorationTextWidget extends StatelessWidget {  
  const DecorationTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
  
        Text(  
          'Underline',  
          style: TextStyle(  
            fontSize: 20,  
            decoration: TextDecoration.underline,  
          ),  
        ),  
  
        SizedBox(height: 15),  
  
        Text(  
          'Line Through',  
          style: TextStyle(  
            fontSize: 20,  
            decoration: TextDecoration.lineThrough,  
          ),  
        ),  
  
        SizedBox(height: 15),  
  
        Text(  
          'Background',  
          style: TextStyle(  

```

```
        fontSize: 20,  
        backgroundColor: Colors.yellow,  
      ),  
    ),  
  
    SizedBox(height: 15),  
  
    Text(  
      'Text Shadow',  
      style: TextStyle(  
        fontSize: 30,  
        shadows: [  
          Shadow(  
            offset: Offset(3, 3),  
            blurRadius: 5,  
          ),  
        ],  
      ),  
    ),  
  ],  
);  
}  
}
```

Ini juga bisa dibuat latihan:

Ubahlah `offset`, `blurRadius`, dan `backgroundColor` kemudian lihat hasilnya.

7. Text Alignment

```
class AlignmentTextWidget extends StatelessWidget {  
  const AlignmentTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: double.infinity,  
      padding: const EdgeInsets.all(10),  
      decoration: BoxDecoration(  
        border: Border.all(),  
      ),  
      child: const Column(  
        children: [  
  
          Text(  
            'Left',  
            textAlign: TextAlign.left,  
          ),  
  
          Text(  
            'Center',  
            textAlign: TextAlign.center,  
          ),  
  
          Text(  
            'Right',  
            textAlign: TextAlign.right,  
          ),  
  
          Text(  
            'Justify: Flutter adalah framework '  
            'untuk membuat aplikasi menggunakan '
```

```
'satu codebase.',  
textAlign: TextAlign.justify,  
),  
],  
),  
);  
}  
}
```

Di sini mahasiswa bisa mencoba:

TextAlign.left

TextAlign.center

TextAlign.right

TextAlign.justify

Catatan penting: `textAlign` akan lebih mudah terlihat efeknya jika `Text` mempunyai lebar tertentu, misalnya berada di dalam `Container` atau `SizedBox`.

8. maxLines dan overflow

```
class OverflowTextWidget extends StatelessWidget {  
  const OverflowTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: double.infinity,  
      padding: const EdgeInsets.all(10),  
      decoration: BoxDecoration(  
        border: Border.all(),  
      ),  
      child: const Text(  
        'Flutter adalah framework UI dari Google '  
        'yang dapat digunakan untuk membuat aplikasi '  
        'Android, iOS, Web dan Desktop.',  
        maxLines: 2,  
        overflow: TextOverflow.ellipsis,  
      ),  
    );  
  }  
}
```

Hasilnya kira-kira:

Flutter adalah framework UI dari Google yang
dapat digunakan untuk membuat aplikasi...

Kemudian mahasiswa dapat mencoba:

maxLines: 3

atau mengganti:

`TextOverflow.ellipsis`

dengan:

`TextOverflow.clip`

9. RichText dan TextSpan

Nah, ini bagus untuk memperkenalkan konsep **nested object** / **children** secara sederhana.

```
class RichTextWidget extends StatelessWidget {  
  const RichTextWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const RichText(  
      text: TextSpan(  
        style: TextStyle(  
          fontSize: 20,  
          color: Colors.black,  
        ),  
        children: [  
          TextSpan(  
            text: 'Saya sedang belajar ',  
          ),  
          TextSpan(  
            text: 'Flutter',  
            style: TextStyle(  
              fontWeight: FontWeight.bold,  
              color: Colors.blue,  
            ),  
          ),  
          TextSpan(  
            text: ' dan ',  
          ),  
          TextSpan(  
            text: 'Dart',  
            style: TextStyle(  
              fontWeight: FontWeight.bold,  
              color: Colors.green,  
            ),  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```
],  
,  
);  
}  
}
```

Hasil:

Saya sedang belajar Flutter dan Dart

^^^^^^^ ^^^^

berbeda style

Ini kemudian bisa dikembangkan menjadi:

Saya sedang belajar Flutter.

Flutter → biru + bold

Saya sedang belajar Dart.

Dart → hijau + bold

10. TextScaler

Untuk materi dasar, saya sarankan contoh ini dibuat sederhana dulu.

```
class TextScalerWidget extends StatelessWidget {  
  const TextScalerWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
  
        Text(  
          'Text normal',  
          style: TextStyle(  
            fontSize: 20,  
          ),  
        ),  
  
        SizedBox(height: 20),  
  
        Text(  
          'Text dengan ukuran lebih besar',  
          textScaler: TextScaler.linear(1.5),  
          style: TextStyle(  
            fontSize: 20,  
          ),  
        ),  
  
      ],  
    );  
  }  
}
```

Konsepnya:

fontSize = 20

TextScaler 1.0



20

TextScaler 1.5



30

Ini bisa dikaitkan dengan **accessibility**: pengguna dapat menggunakan ukuran teks sistem yang lebih besar.

11. SectionTitle

Widget kecil ini digunakan supaya kode utama lebih rapi.

```
class SectionTitle extends StatelessWidget {  
  final String title;  
  
  const SectionTitle({  
    super.key,  
    required this.title,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(  
      title,  
      style: const TextStyle(  
        fontSize: 22,  
        fontWeight: FontWeight.bold,  
      ),  
    );  
  }  
}
```

Di sini mahasiswa sekaligus mulai belajar **class + constructor + parameter**:

```
final String title;  
  
const SectionTitle({  
  super.key,  
  required this.title,  
});
```

Buatkan Masing masing lebih dalam tentang Text dan buat masing masing untuk Class widget

1. Kustomisasi `TextStyle` yang Lebih Mendalam

- **Font & Ukuran:** Mengatur jenis huruf (`fontFamily`), ketebalan (`fontWeight` seperti `bold`, `w600`), dan kemiringan (`fontStyle`).
- **Spasi:** Mengatur jarak antarhuruf (`letterSpacing`), jarak antarkata (`wordSpacing`), dan tinggi baris (`height`).
- **Dekorasi & Efek:** Menambahkan garis bawah/coret (`decoration` dengan `TextDecoration.underline` atau `lineThrough`), warna latar belakang (`backgroundColor`), hingga efek bayangan (`shadows`).

2. Pengaturan Tata Letak & Perataan (`Layout & Alignment`)

- **Perataan Teks (`textAlign`):** Mengatur posisi teks menjadi rata kiri, kanan, tengah, atau rata kiri-kanan (`justify`).
- **Batasan Baris (`maxLines`) & Overflow (`textOverflow`):** Menentukan jumlah maksimum baris yang ditampilkan dan bagaimana teks yang terlalu panjang ditangani (misalnya dipotong dengan titik-titik `TextOverflow.ellipsis`).
- **Soft Wrap (`softWrap`):** Mengatur apakah teks harus otomatis turun ke baris berikutnya jika melebihi lebar layar.

3. Menggabungkan Berbagai Gaya dalam Satu Paragraf (`RichText & TextSpan`)

- Jika Anda ingin membuat satu kalimat di mana sebagian kata dicetak tebal, berwarna berbeda, atau bisa diklik (seperti tautan), Anda tidak bisa hanya menggunakan `Text` biasa, melainkan menggunakan `RichText` dengan anak-anak `TextSpan`.

4. Skala & Aksesibilitas (`TextScaler`)

- Mempelajari cara teks beradaptasi dengan pengaturan ukuran font sistem perangkat pengguna untuk mendukung aksesibilitas (penyandang disabilitas atau pengguna dengan preferensi teks besar).